



NOWITNESS

Penetration Test Report

Lace Browser Extension Security Assessment Round 2

Date	February 27, 2026
Client	Input Output (IO)
Project	Lace Browser Extension
Version	2.25.0

Contents

- Disclosure 1**
- Disclaimer 2**
- Scope 3**
- Assessment Overview 4**
- Assessment Components 5**
- Executive Summary 6**
- Code Base 7**
- Severity Classification 8**
- Finding Severity Ratings 9**
- Findings 10**
 - [L 201] Plaintext Secrets Stored in Redux State During Onboarding/Restore 11
 - [L 202] Decrypted recovery phrase not zeroized after use 13
 - [L 203] Dust secret key not cleared on idle/lock for Midnight accounts 14
 - [L 204] Shared Provider Credentials in Client Traffic Can Be Abused 15
 - [I 101] No Authentication Required to Open Extension or View Wallet Data 18
 - [I 102] No Mechanism for Detecting or Filtering Spam Tokens 20
 - [I 103] Vulnerable Elliptic Package in Transitive Dependencies (CVE-2025-14505) . 22

Disclosure

This document contains proprietary information belonging to No Witness Labs. Duplication, redistribution, or use, in whole or in part, in any form, requires prior written consent from No Witness Labs.

Nonetheless, both the customer **Input Output (IO)** and No Witness Labs are authorized to share this document with the public to demonstrate security compliance and transparency regarding the outcomes of the Protocol.

Disclaimer

This review reflects the state of the codebase at the time of assessment and should be regarded as a point-in-time evaluation. Any changes made after the assessment period fall outside the scope of this report.

Although every effort was made to identify potential risks and vulnerabilities, no security review can guarantee the discovery of all issues. The information and recommendations in this report are provided solely for technical purposes and must not be considered investment advice.

To strengthen security assurance, projects are encouraged to seek multiple independent reviews and to supplement audits with practices such as bug bounty programs.

The scope of this penetration test was limited to the Lace browser extension as deployed in the provided test environment and did not include the underlying blockchain, backend services, host browser or operating system, or any external dApps or websites.

Certain functional limitations in the test environment restricted end-to-end validation of in-scope Midnight dApp-connector connection and authorization flows.

Scope

In-Scope Components

- Browser Extension (popup, background scripts, content scripts)
- Web Wallet Interface (if applicable)
- APIs and Network Communication
- Storage (localStorage, chrome.storage, IndexedDB)
- Authentication and Session Handling
- Password / Seed-phrase Handling Modules
- Domains and Subdomains
- IP Addresses

Out-of-Scope Components

- Third-party dApps, unless whitelisted for integration tests
- Remote and local proof server
- Remote node RPC server
- Remote indexer

Assessment Overview

From **January 26, 2026** to **February 27, 2026**, **Input Output (IO)** engaged No Witness Labs to conduct a comprehensive security assessment of the **Lace Browser Extension** protocol. The evaluation was performed in accordance with established industry best practices for code auditing and protocol analysis.

The audit followed a structured five-phase process:

- **Planning** – Defined the assessment scope and objectives, and aligned expectations with the client.
- **Discovery** – Acquired an understanding of the protocol through client discussions, documentation review, and preliminary analysis of architecture and design.
- **Manual Revision** – Conducted an in-depth review of the codebase to identify potential vulnerabilities, design weaknesses, and security risks.
- **Reporting** – Documented all confirmed findings, assigned severity levels, and provided actionable recommendations for remediation.

Assessment Components

Manual review

Our manual review covers the Lace browser extension, and supporting APIs. It focuses on a wide range of attack surfaces and failure modes defined in the agreed test plan, including but not limited to:

- Web-facing security (XSS, CSRF, origin checks, malicious content)
- Extension manifest scope, host permissions, and content-script scoping
- Storage security for mnemonics, private keys, and passwords (EMIP-3 alignment)
- Authentication and session management (unlock flows, timeouts, persistence)
- Password strength and offline-guess resistance
- Seed phrase generation
- Transaction preview versus signed-transaction integrity
- Message passing between in-page, content, and background scripts
- Network/API security (remote node/indexer usage, TLS/MITM resilience)
- Telemetry, logging, and analytics data minimization

Executive Summary

Project Overview

Lace Browser Extension v2.25 is an open-source, non-custodial light wallet built by Input Output (IO) and distributed as a Chrome-compatible browser extension. It enables self-custody management of digital assets and supports connecting to Web3 applications via an in-browser dApp-connector flow.

This release is a multi-network wallet for Cardano, Bitcoin, and Midnight. It includes a range of features such as wallet creation and import, transaction signing, dApp connectivity, and support for multiple accounts and networks. The extension is designed with a focus on security, usability, and performance.

Code Base

Repository

<https://github.com/input-output-hk/lace-platform.git>

Commit

380484d1594a93965f82525bdb109ee6d9c30834

Severity Classification

- **Critical:** Vulnerabilities that enable direct theft of assets, permanent loss of funds, or catastrophic protocol failure. These represent immediate and severe financial risk with straightforward exploitation paths.
- **High:** Vulnerabilities that can result in financial loss or significant harm to users or the protocol, though impact may be limited to specific functions or require certain conditions. Exploitation paths exist but may be more complex than Critical Vulnerabilities.
- **Medium:** Vulnerabilities that do not directly result in financial loss but may temporarily impair functionality or create exploitable conditions. These are typically limited to state manipulations and include issues such as front-running susceptibility or logic flaws that compromise transaction integrity.
- **Low:** Low-impact Vulnerabilities with limited attack vectors, minimal attacker incentive, or requiring highly specific scenarios. These typically do not result in financial loss or significant harm to users or the protocol.
- **Informational:** Findings without immediate security or financial implications. These include optimization opportunities, code quality improvements, inconsistencies in naming conventions, design suggestions, and gaps in documentation.

Impact–Likelihood Severity Matrix

	High likelihood	Medium likelihood	Low likelihood
High impact	C	H / M	M
Medium impact	H	M	M / L
Low impact	M	M / L	L

Finding Severity Ratings

The following table defines levels of severity and score range that are used throughout the document to assess vulnerability and risk impact

	Level	Severity	Count
	5	Critical	0
	4	High	0
	3	Medium	0
	2	Low	4
	1	Informational	3
Total Findings			7

Findings

ID	Title	Severity	Status
201	Plaintext Secrets Stored in Redux State During Onboarding/Restore	Low	Pending
202	Decrypted recovery phrase not zeroized after use	Low	Pending
203	Dust secret key not cleared on idle/lock for Midnight accounts	Low	Pending
204	Shared Provider Credentials in Client Traffic Can Be Abused	Low	Pending
101	No Authentication Required to Open Extension or View Wallet Data	Informational	Pending
102	No Mechanism for Detecting or Filtering Spam Tokens	Informational	Pending
103	Vulnerable Elliptic Package in Transitive Dependencies (CVE-2025-14505)	Informational	Pending

[L 201] Plaintext Secrets Stored in Redux State During Onboarding/Restore

Level	Severity	Status
2	Low	Pending

Category: Extension & Storage Security / Secret Handling

Affected Files:

- packages/contract/onboarding-v2/src/store/slice.ts
- packages/contract/account-management/src/store/slice.ts
- packages/module/onboarding/src/pages/OnboardingDesktopLogin/useOnboardingDesktopLogin.ts
- packages/module/onboarding/src/pages/OnboardingCreateWallet/useOnboardingCreateWallet.ts

Affected Components: Onboarding flow, restore flow, Redux state

Description

During onboarding and wallet restore, the app stores the user's plaintext password and recovery phrase inside Redux state. Even if the data is cleared on success, it can persist across error paths or navigation paths that do not explicitly clear it. Because Redux is a global, long-lived, and highly observable state container, keeping plaintext secrets there materially increases the exposure surface. This is a defense-in-depth issue: it does not imply secrets are exfiltrated by default, but it widens the set of components and tools that can access them while they are resident.

This breaks the project's general pattern of persisting only encrypted key material (e.g., `encryptedRecoveryPhrase`, `encryptedRootPrivateKey`, `encryptedSeed`) and keeping plaintext data ephemeral. The net effect is a broader, longer-lived exposure of the most sensitive material, and a higher likelihood that debugging or inspection pathways capture secrets.

Evidence

Plaintext secrets are stored in Redux state during onboarding/restore:

- packages/contract/onboarding-v2/src/store/slice.ts
- packages/contract/account-management/src/store/slice.ts
- packages/module/onboarding/src/pages/OnboardingDesktopLogin/useOnboardingDesktopLogin.ts
- packages/module/onboarding/src/pages/OnboardingCreateWallet/useOnboardingCreateWallet.ts

Concrete fields:

- `state.onboardingV2.pendingCreateWallet.password`
- `state.onboardingV2.pendingCreateWallet.recoveryPhrase`
- `state.accountManagement.restoreWalletFlow.passphrase`
- `state.accountManagement.restoreWalletFlow.recoveryPhrase`

Impact

Plaintext recovery phrases and passwords grant full control of a wallet. Storing them in Redux:

- Increases the access surface to any module, middleware, or tooling with store access.
- Extends lifetime beyond a single screen unless every failure/cancel path clears state.
- Creates a higher risk of accidental logging or inspection artifacts containing secrets.

Recommendation

- Avoid storing plaintext secrets in Redux; use local component state or a short-lived in-memory buffer scoped to the minimal set of components.
- Clear on all failure/cancel paths; ensure back/cancel and error transitions wipe any secret-bearing state.
- Prefer byte arrays or redacted types where possible, and zeroize after use.

Proof of Concept

1. Open the extension UI during onboarding/restore.
2. In extension UI devtools, read Redux state and extract:
 - `state.onboardingV2.pendingCreateWallet.password`
 - `state.onboardingV2.pendingCreateWallet.recoveryPhrase`
 - `state.accountManagement.restoreWalletFlow.passphrase`
 - `state.accountManagement.restoreWalletFlow.recoveryPhrase`
3. Observe plaintext secret values.

[L 202] Decrypted recovery phrase not zeroized after use

	Level	Severity	Status
	2	Low	Pending

Category: Secret Handling / Memory Hygiene

Affected File: `packages/contract/recovery-phrase/src/utils.ts`

Description `decryptRecoveryPhraseMnemonic` converts the decrypted `ByteArray` into a UTF-8 string (`mnemonicString`) and splits it into word arrays without wiping the intermediate buffers. This leaves additional plaintext copies of the recovery phrase in memory beyond the returned `mnemonicWords`.

Impact While JavaScript memory is inherently difficult to reliably wipe, leaving extra plaintext copies increases exposure in the event of memory inspection or unintended logging. This is a defense-in-depth issue but relevant for keystore/secret handling.

[L 203] Dust secret key not cleared on idle/lock for Midnight accounts

	Level	Severity	Status
	2	Low	Pending

Category: Secret Handling / Key Lifecycle

Affected Files:

- `packages/module/blockchain-midnight/src/store/side-effects/watch.ts`
- `packages/module/blockchain-midnight/src/store/dependencies.ts`

Description Midnight account key management intentionally leaves dust key material in memory:

- `AccountKeys.clear()` zeroizes `nightKeyBuffer` and `zswapKeyBuffer`, but does not clear `dustKeyBuffer` or `dustSecretKey` (TODO comment).
- The dust wallet is started once and never stopped, and the code notes that the dust secret key “stays in memory until account watcher stops.”

This means the idle timeout in `createAccountKeyManager` does not fully clear all key material for Midnight accounts.

Impact Prolonged in-memory residency of dust private key material increases exposure in the event of memory inspection or compromise of the extension process. This is a defense-in-depth issue but relevant to keystore/secret handling expectations.

[L 204] Shared Provider Credentials in Client Traffic Can Be Abused

Level	Severity	Status
2	Low	Pending

Category: Infrastructure & API Security / Credential Exposure

Affected Files:

- `lace-platform/packages/module/cardano-provider-blockfrost/src/store/dependencies.ts`
- `lace-platform/packages/module/bitcoin-provider-maestro/src/store/dependencies.ts`
- `lace-platform/apps/lace-extension/src/util/config.ts`
- `lace-platform/apps/lace-extension/webpack/.env.defaults`
- `lace-extension-2.25.0/expo/_expo/static/js/web/index-901ebe8de94565f95c3137afd1cb53bc.js`
- `lace-platform/packages/module/posthog-client-extension/src/store/dependencies.ts`
- `lace-extension-2.25.0/expo/_expo/static/js/web/init-5f2df6f939f8f2f9536c1f4f95569a55.js`

Affected Components: Cardano Blockfrost provider, Bitcoin Maestro provider path, PostHog telemetry integration

Description

Lace sends provider credentials from extension-side requests. In the current build, Blockfrost requests carry a reusable `project_id` value in the header, and that value can be extracted and replayed outside the wallet. A similar `api-key` pattern exists for the Maestro Bitcoin provider path (the constants are `notused` in this artifact, but the design supports the same exposure when enabled).

This is not necessarily just an exposure issue. The practical risk is that third parties can burn shared provider quota and trigger provider throttling, which can degrade wallet reliability for real users. The PostHog token sent to `e.lw.io` appears to be a normal browser analytics token and is lower risk, but it still exposes integration metadata in client traffic.

- Primary risk (availability/DoS): replayed credentials consume shared limits and cause degraded or failed provider-backed operations.
- Secondary risk (confidentiality): visible credentials are exposed.

Evidence

- Blockfrost requests include client-supplied `project_id` header:
 - `lace-platform/packages/module/cardano-provider-blockfrost/src/store/dependencies.ts`
- Maestro requests include client-supplied `api-key` header:
 - `lace-platform/packages/module/bitcoin-provider-maestro/src/store/dependencies.ts`
- Runtime config wires provider IDs and telemetry values from env:
 - `lace-platform/apps/lace-extension/src/util/config.ts`
 - `lace-platform/apps/lace-extension/webpack/.env.defaults`

- Packaged build includes concrete Blockfrost IDs and PostHog token/url constants:
 - `lace-extension-2.25.0/expo/_expo/static/js/web/index-901ebe8de94565f95c3137afd1cb53bc.js`
- Packaged build includes Maestro constants, currently set to `notused` in this artifact:
 - `lace-extension-2.25.0/expo/_expo/static/js/web/index-901ebe8de94565f95c3137afd1cb53bc.js`
- PostHog initializes to IOG-owned endpoint:
 - `lace-platform/packages/module/posthog-client-extension/src/store/dependencies.ts`
 - `lace-extension-2.25.0/expo/_expo/static/js/web/init-5f2df6f939f8f2f9536c1f4f95569a55.js`

Example request that was captured:

1	POST /api/v0/tx/submit HTTP/2	http
2	Host: cardano-preprod.blockfrost.io	
3	Cookie: __cflb=0H28ur4xJGQN715a2kk86EKJhXTCVn62YbFeU7NsXpH	
4	Content-Length: 354	
5	User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36	
6	Content-Type: application/cbor	
7	Project_id: preprod##### <-- Shared credential sent in client requests	
8	Accept: */*	
9	Origin: chrome-extension://lfceaaafbmbnkjadndocfbbdeencnnapi	
10	Sec-Fetch-Site: none	
11	Sec-Fetch-Mode: cors	
12	Sec-Fetch-Dest: empty	
13	Sec-Fetch-Storage-Access: active	
14	Accept-Encoding: gzip, deflate, br	
15	Accept-Language: en-US,en;q=0.9	
16	Priority: u=1, i	

Impact

- Unauthorized replay can consume shared provider quota and push traffic into provider throttling behavior.
- Users can experience practical DoS symptoms in wallet flows that depend on those providers (timeouts, stale data, delayed refresh, intermittent failures).
- Direct client-to-provider traffic also increases request volume that could be reduced with proxy-side caching.

Recommendation

- Proxy third-party provider calls (Blockfrost and Maestro) through an IOG-controlled backend.

- Keep provider credentials server-side and avoid shipping reusable third-party keys in the client bundle.
- Add abuse controls at the proxy (rate controls, anomaly detection, and basic per-user/IP shaping).
- Add proxy-side caching for common reads to cut upstream load and improve wallet responsiveness.
- Continue using the IOG-owned analytics endpoint and minimize direct browser-to-third-party credentialed requests where practical.
- Rotate currently deployed provider credentials and monitor for unusual consumption.

[I 101] No Authentication Required to Open Extension or View Wallet Data

Level	Severity	Status
1	Informational	Pending

Category: Authentication & Session Management

Affected Files:

- `packages/contract/authentication-prompt/src/store/state-machine.ts`
- `packages/contract/tx-executor/src/store/side-effects.ts:177`
- `packages/contract/recovery-phrase/src/store/side-effects.ts:78`
- `packages/contract/account-management/src/store/side-effects.ts:370,622,719,825`

Description

The Lace browser extension does not require authentication when the popup is opened. Any user with access to the browser can open the extension and immediately view wallet balances, addresses, transaction history, and the address book without entering credentials.

Authentication is only triggered for signing transactions (`tx-executor`), exporting the recovery phrase (`recovery-phrase`), and account management operations (`account-management`). These flows invoke the authentication prompt contract (`packages/contract/authentication-prompt/`), which manages password and biometric authentication through a state machine (`Idle -> Preparing -> OpenPassword/OpenBiometric -> Verifying -> Completing`).

There is no idle timeout, auto-lock, or session lock mechanism that restricts access to the extension popup after a period of inactivity.

Impact

A user who steps away from their computer with the browser open exposes wallet information to anyone with physical access. Account balances across all supported blockchains (Cardano, Bitcoin, Midnight), wallet addresses, full transaction history, and the address book (saved contacts) are viewable without authentication.

Funds cannot be transferred without the password, as signing still requires authentication.

Steps to Reproduce

1. Install the Lace browser extension and set up a wallet with a password.
2. Close the extension popup.
3. Reopen the extension popup without restarting the browser.
4. Observe that balances, addresses, transaction history, and address book are displayed without any authentication prompt.
5. Repeat after leaving the browser idle for an extended period (e.g., 30 minutes). Observe the same behavior. No lock screen or re-authentication is required.

Recommendation

1. Add an optional auto-lock feature that requires password or PIN entry after a configurable idle period (e.g., 5, 15, 30 minutes).
2. Add an option to require authentication when the extension popup is opened, configurable by the user in settings.

Evidence

- `packages/contract/authentication-prompt/src/store/state-machine.ts`: authentication state machine. Only transitions from `Idle` to `Preparing` when `requested` is dispatched by signing or export operations.
- `packages/contract/tx-executor/src/store/side-effects.ts:177`: `authPrompt()` invoked during transaction execution.
- `packages/contract/recovery-phrase/src/store/side-effects.ts:78`: `authPrompt()` invoked during recovery phrase export.
- `packages/contract/account-management/src/store/side-effects.ts:370,622,719,825`: `authPrompt()` invoked for account management operations.
- No references to `authPrompt`, `lockScreen`, `autoLock`, or `idleTimeout` exist in the extension popup initialization or routing logic.

[I 102] No Mechanism for Detecting or Filtering Spam Tokens

	Level	Severity	Status
	1	Informational	Pending

Category: Web-Facing Security

Affected Files:

- `packages/module/dapp-connector-cardano/src/common/utils/asset-utils.ts:106-131`
- `packages/contract/tokens`

Description

Lace does not implement spam token detection or filtering. The `getAssetDisplayInfo` function (`packages/module/dapp-connector-cardano/src/common/utils/asset-utils.ts:106`) retrieves token metadata from the store and returns display information (name, ticker, image, decimals, NFT status) without checking against any token registry, verification database, or blocklist.

On Cardano, anyone can mint native tokens and send them to arbitrary addresses at minimal cost (approximately 2 ADA per recipient). All received tokens appear in the wallet balance with no distinction between verified and unverified tokens.

Impact

Users can receive unsolicited tokens that appear in their wallet alongside legitimate holdings:

1. Fake tokens can use names and logos that resemble legitimate projects, potentially misleading users about the value of their holdings.
2. NFT metadata can contain links that direct users to phishing sites.
3. Users have no in-wallet signal to distinguish verified tokens from unverified ones, increasing the risk of interacting with malicious dApps promoted through airdropped tokens.

Steps to Reproduce

1. Open Lace and navigate to the token list on a Cardano wallet that has received unsolicited tokens.
2. Observe that all tokens are displayed with equal presentation. No verification badge, warning, or filtering is applied.
3. Review `getAssetDisplayInfo` in `asset-utils.ts:106`. Observe that it returns metadata directly from the store without any verification check. The `StoredTokenMetadata` type does not include a verification status field.

Recommendation

1. Integrate the Cardano Token Registry to display verification status for known tokens.
2. Add visual indicators distinguishing verified tokens from unverified ones.
3. Display a warning when users attempt to interact with unverified tokens in transactions.

Evidence

- `packages/module/dapp-connector-cardano/src/common/utils/asset-utils.ts:106-131`: `getAssetDisplayInfo` returns metadata without verification status.
- `packages/contract/tokens` (`StoredTokenMetadata` type): no field for verification status or spam classification.
- [Cardano Token Registry](#)

[I 103] Vulnerable Elliptic Package in Transitive Dependencies (CVE-2025-14505)

Level	Severity	Status
1	Informational	Pending

Category: Code Quality & Communication Security

Affected Files:

- package.json
- package-lock.json
- packages/module/blockchain-bitcoin/src/wallet/BitcoinSigner.ts

Description

The `elliptic` npm package (v6.6.1) is present in Lace as a transitive dependency through the Trezor hardware wallet integration. CVE-2025-14505 (published January 8, 2026) identifies a flaw in the ECDSA signature implementation: the `_truncateToN` function in `ec/index.js` incorrectly calculates the byte length of the nonce `k` when the value has leading zeros. This produces faulty signatures where the nonce is linearly correlated with the correct nonce, which could allow an attacker to recover private keys if they obtain both a faulty and a correct signature generated by the same key.

The vulnerability is present in all published versions of `elliptic` through v6.6.1. No upstream fix has been released. The `elliptic` repository has not received updates since November 2024 despite multiple open security issues (#321, #340, #341, #344).

The dependency enters Lace through two paths:

1	@cardano-sdk/wallet (v0.53.21, direct dependency)
2	-> @cardano-sdk/hardware-trezor (~0.7.29)
3	-> @trezor/connect (9.4.0)
4	-> @trezor/blockchain-link (2.3.0)
5	-> ripple-lib (^1.10.1, deprecated)
6	-> ripple-keypairs (1.3.1)
7	-> elliptic (^6.5.4, resolved to 6.6.1)
8	-> @trezor/utxo-lib (2.2.0)
9	-> tiny-secp256k1 (^1.1.6)
10	-> elliptic (^6.4.0, resolved to 6.6.1)

Impact

Lace's own Bitcoin and Cardano signing operations are not directly affected. Lace uses `@bitcoinerlab/secp256k1` (which depends on `@noble/curves`, not `elliptic`) for Bitcoin signing, as seen in `packages/module/blockchain-bitcoin/src/wallet/BitcoinSigner.ts`. Cardano signing uses the Cardano SDK's own cryptographic primitives. Trezor hardware wallet signing happens on the device itself, not through JavaScript.

The risk to Lace is indirect:

1. The `elliptic` package is loaded into the application bundle through Trezor integration. If any code path in `@trezor/utxo-lib` or `@trezor/blockchain-link` invokes `elliptic` for ECDSA operations (such as address derivation, script validation, or key verification), those operations would use the vulnerable implementation.
2. The `ripple-lib` dependency in the second chain is deprecated and pulls `elliptic` for XRP keypair operations. Even though Lace does not support XRP, the vulnerable code is included in the dependency tree and bundled into the application.
3. The presence of a known-vulnerable cryptographic library in a cryptocurrency wallet's dependency tree is a supply chain risk. Future code changes or dependency updates could inadvertently route signing operations through `elliptic`.

Steps to Reproduce

1. Examine `package.json` line 58: `"@cardano-sdk/wallet": "0.53.21"`
2. Examine `package-lock.json` line 6576: `wallet` depends on `"@cardano-sdk/hardware-trezor": "~0.7.29"`
3. Trace the first dependency chain: `hardware-trezor` -> `@trezor/connect` -> `@trezor/blockchain-link` -> `ripple-lib` -> `ripple-keypairs` -> `elliptic`
4. Trace the second dependency chain: `hardware-trezor` -> `@trezor/connect` -> `@trezor/utxo-lib` -> `tiny-secp256k1` -> `elliptic`
5. Confirm `elliptic` v6.6.1 resolved at `package-lock.json` line 49896

Recommendation

1. File an issue with `@cardano-sdk` maintainers to update `@trezor/connect` to a version that removes the `elliptic` dependency.
2. File an issue with Trezor to replace `elliptic` in their packages:
 - Replace `tiny-secp256k1` with `@noble/secp256k1` in `@trezor/utxo-lib`
 - Replace the deprecated `ripple-lib` with `xrpl.js` in `@trezor/blockchain-link`
3. Monitor for upstream updates. The community recommends migrating to [\[@noble/curves\]](#) or using [\[elliptic-to-noble\]](#) as a drop-in shim.
4. Consider making Trezor hardware wallet support an optional dependency to reduce attack surface for users who do not use Trezor devices.

Evidence

- `package.json` line 58: `"@cardano-sdk/wallet": "0.53.21"`
- `package-lock.json` line 6576: `"@cardano-sdk/hardware-trezor": "~0.7.29"`
- `package-lock.json` line 26557: `"ripple-lib": "^1.10.1"` in `@trezor/blockchain-link`
- `package-lock.json` line 26834: `"tiny-secp256k1": "^1.1.6"` in `@trezor/utxo-lib`
- `package-lock.json` line 76654: `"elliptic": "^6.5.4"` in `ripple-keypairs`
- `package-lock.json` line 81881: `"elliptic": "^6.4.0"` in `tiny-secp256k1`
- `package-lock.json` line 49896: `elliptic` resolved to v6.6.1

- `BitcoinSigner.ts` line 1: Lace Bitcoin signing imports `@bitcoinerlab/secp256k1`, not `elliptic`
- [CVE-2025-14505 \(HeroDevs\)](#)
- [Snyk Advisory SNYK-JS-ELLIPTIC-14908844](#)
- [GitHub Issue indutny/elliptic#321](#) (original report by Daniel Bleichenbacher, October 13, 2024)
- [Trezor Suite Issue #18334](#): "Update elliptic lib in everstake sdk," closed April 2025 via PR #18385. This addressed elliptic in the Everstake SDK but does not resolve CVE-2025-14505, as no fix exists in elliptic itself.
- [Cardano-JS-SDK PR #1434](#): Dependabot PR to bump elliptic from 6.5.4 to 6.5.7, closed without merge in August 2024, superseded by PR #1435.
- [Moving Beyond the NPM elliptic Package \(Soatok, Nov 2025\)](#)

End of Report



No Witness Labs — Confidential

Copyright © 2026